

Week 8 — Lecture Notes (Instructor)

Error Handling & User Feedback

Phase 3: Interface Binding · AI Zone: ON — required tool, prompt logs mandatory Session length: ~90 min lecture + handoff to lab **Milestone this week:** Deliverable 3 (Interface & View Binding, 30%) due end of session **Stack:** Concepts and pseudocode primary; a Blade example included.

Learning Objectives

By the end of this session, students can:

1. Explain why graceful failure is core engineering, not polish.
2. Identify the kinds of failure (422, 404, 500, network) and handle each correctly.
3. Write helpful, specific error messages in the interface's voice.
4. Implement consistent feedback — loading, success, and error — across the whole app.
5. Deliberately test failure paths, not just the happy path.

Session Outline

Time	Segment
0:00-0:06	Recap: forms send data — now handle what goes wrong
0:06-0:22	Part A — Failure is normal
0:22-0:38	Part B — The three states, everywhere
0:38-0:41	Discussion break 1
0:41-0:57	Part C — Writing helpful error messages
0:57-1:12	Part D — Feedback patterns
1:12-1:14	Discussion break 2
1:14-1:24	Part E — Consistency across the app
1:24-1:30	Handoff to lab + Deliverable 3

Recap & Framing (6 min)

Last week your forms learned to send data and handle the happy path. This week you handle everything else — because in a real app, requests fail all the time. A failed action that looks like *nothing happened* is worse than a crash: the user doesn't know what to do. Errors are moments for **direction**, not decoration. AI stays on; keep logging prompts.

Part A — Failure Is Normal (16 min)

Every request can fail, and each kind of failure needs a different response. Teach the four categories students will actually hit:

- **Validation error (422)** — the user sent bad data. *Show the field-level errors* your controllers already return (Week 4/5).
- **Not found (404)** — the record doesn't exist (bad id, deleted item). *Show a clear "not found" state*, not a blank screen.
- **Server error (500)** — your code threw. *Show a general "something went wrong" message* — never the raw stack trace.
- **Network error** — the request never completed (offline, timeout). *Show a retry option*.

The principle: the client must handle the *whole* set of responses its controllers can return — not just 201/200. If a branch isn't handled, that failure is invisible to the user.

Part B — The Three States, Everywhere (16 min)

Every asynchronous action lives in one of three states, and each needs a visible UI:

└─ loading

└─ success

└─ error

→ show a spinner / disable the control

→ confirm it; update the UI

→ show what went wrong and what to do

This is the async lifecycle from Week 7 — now applied to every action in the app, not just the create form. A delete, an edit, a page load: each one loads, then succeeds or fails.

The tell of an amateur app: some actions give feedback and others silently do nothing. Consistency is the lesson.

Discussion break 1: "You click Delete and nothing visibly happens for two seconds. What three things should the UI have shown?" (Loading → then success or error.)

Part C — Writing Helpful Error Messages (16 min)

An error message is design material, not an afterthought. Good messages are **specific, actionable, and in the interface's voice**.

Bad	Good
"Error 422"	"Quantity must be at least 1."
"Something went wrong" (for a validation error)	"Please choose a status: pending, paid, or shipped."
A raw stack trace	"We couldn't save your order. Try again."
"Invalid input"	"Item name can't be longer than 100 characters."

Rules to teach:

- Say *what* went wrong and *how to fix it*.
- Name the field when it's a field problem (you already have this from the 422 shape).
- Don't blame the user; don't apologize theatrically; don't leak internals.
- Match the app's voice — plain, calm, sentence case.

This connects straight to Week 4/5: because your controllers return a standardized error with a message and field, the UI has exactly what it needs to write a good message.

Part D — Feedback Patterns (15 min)

A small toolbox students will reuse everywhere:

- **Inline field errors** — the message next to the field that caused it (best for 422 validation).
- **Toast / banner** — a transient or top-of-page message for global success/failure ("Order saved," "Couldn't connect").
- **Empty states** — a helpful message and a next action when there's no data (from the Week 2 wireframes).
- **Loading indicators** — spinner, skeleton, or a disabled control with a label ("Saving...").
- **Disabled controls while pending** — prevents double-submits (Week 7).
- **Confirmation for destructive actions** — the "are you sure?" before a delete.

Match the pattern to the situation: field errors inline, global outcomes as a toast, missing data as an empty state.

Discussion break 2: "Where does a 422 validation error belong — a toast, or inline by the field?" (Inline — it points the user to the exact fix.)

Part E — Consistency Across the App (10 min)

The difference between a demo and a product is that a product behaves **the same way everywhere**. One shared approach to loading, one to errors, one to success — reused on every screen. If create shows a spinner but edit doesn't, users feel it even if they can't name it.

Encourage teams to define their feedback approach once (a shared component or helper) and apply it across all CRUD actions. This is also where the standardized response shape pays off again: uniform responses make uniform feedback easy.

In Laravel (one concrete example)

Blade surfaces validation errors and flash messages directly:

```
{{-- inline field error --}}
<input name="qty" value="{{ old('qty') }}">
@error('qty') <span class="field-error">{{ $message }}</span> @enderror

{{-- global flash message after a redirect --}}
@if (session('status'))
```

```
<div class="toast">{{ session('status') }}</div>
@endif
```

`@error` renders the exact message your Form Request produced; `session('status')` carries a one-time confirmation. The concept — inline errors for fields, a toast for global outcomes — transfers to React/Vue/Angular.

Handoff to Lab + Deliverable 3 (6 min)

Teams add loading indicators and error handling across every action, rewrite vague messages to be helpful, and make the whole app behave consistently — then assemble **Deliverable 3 (Interface & View Binding, 30%)**: a fully interactive local application where CRUD operates seamlessly. Point them to the Deliverable 3 spec for the Definition of Done.

Instructor Notes / Common Pitfalls

- **The silent failure is the #1 problem.** Make teams prove every failure is *visible*. If breaking the network shows nothing, it fails.
- **Only the happy path handled.** Teams wire success and forget the 422/500/network branches. Require all of them.
- **Technical messages leak through.** "422" and stack traces reaching the user are point-losers. Rewrite to human.
- **Inconsistency.** Walk the app: does every action load, confirm, and fail the same way? If not, it's not done.
- **Destructive actions without confirmation.** A one-click delete with no "are you sure?" is a real bug.
- **Prompt logs.** Error-handling scaffolds are common AI asks — reviewed and logged. Deliverable 3 grades the log.